

Documentazione: Sistema di Gestione dell'Orario delle Lezioni

Alessandro Mormone, Claudio Mona, Alfredo Lucco
Gruppo 35

Indice

1. Introduzione	2
2. Architettura dei Package	2
3. Il Package <i>model</i>	3
3.1 Gestione degli Utenti	3
3.2 Gestione Didattica	3
3.3 Gestione dei Vincoli e delle Richieste	3
4. Il Package <i>controller</i>	4
5. Il Package <i>gui</i>	4
5.1 Schermate Principali	5
5.2 Schermate Operative	5
6. Il Package <i>dao</i>	5
7. Il Package <i>implementazioneDao</i>	6
8. Il Package <i>database_connection</i>	6
9. Funzionalità Principali	6
10. Documentazione Interna del Codice	7
11. Qualità del Codice	7
12. Diagrammi di Sequenza	8
13. Conclusioni	8

1. Introduzione

Il presente documento descrive l'architettura software del progetto "Sistema di Gestione dell'Orario delle Lezioni". L'applicazione è stata sviluppata in Java e utilizza un'interfaccia grafica realizzata con Swing. Lo scopo del sistema è permettere la gestione e la consultazione dell'orario settimanale delle lezioni di un corso di laurea.

Il sistema prevede tre principali tipologie di utenti: studente, docente e docente responsabile. Lo studente può accedere alla propria area riservata e visualizzare l'orario delle lezioni. Il docente può visualizzare il proprio orario e inviare una richiesta di spostamento lezione. Il docente responsabile, invece, può visualizzare le richieste di spostamento ricevute e decidere se approvarle o rifiutarle.

Il progetto è stato organizzato in package distinti, in modo da separare le responsabilità delle varie parti del sistema: modello del dominio, interfaccia grafica, controllo del flusso applicativo, accesso ai dati e connessione al database.

2. Architettura dei Package

L'applicazione è suddivisa nei seguenti package principali:

- *model*: contiene le classi che rappresentano le entità principali del dominio applicativo, come utenti, docenti, studenti, lezioni, aule, insegnamenti, vincoli e richieste di spostamento.
- *gui*: contiene le classi Swing che realizzano l'interfaccia grafica dell'applicazione.
- *controller*: contiene le classi che gestiscono il flusso dell'applicazione e collegano la GUI alla logica applicativa.
- *dao*: contiene le interfacce che definiscono le operazioni di accesso ai dati.
- *implementazione dao*: contiene le implementazioni concrete delle interfacce DAO, basate su database PostgreSQL.
- *database_connection*: contiene la classe responsabile della connessione al database.

Questa organizzazione consente di mantenere separata la logica di dominio dalla grafica e dall'accesso al database, rendendo il progetto più ordinato, leggibile e facilmente modificabile.

3. Il Package model

Il package model rappresenta il cuore del sistema, poiché contiene le classi che modellano gli elementi principali dell'orario universitario.

3.1 Gestione degli Utenti

- **Utente:** classe astratta che rappresenta un utente generico del sistema. Contiene le informazioni comuni a tutti gli utenti registrati, come nome, cognome, email e password. Non viene istanziata direttamente, ma viene utilizzata come classe base per utenti più specifici.
- **Studente:** rappresenta uno studente registrato nel sistema. Estende Utente e aggiunge informazioni specifiche come matricola e anno di corso. L'anno di corso è utile per permettere allo studente di visualizzare l'orario relativo al proprio percorso.
- **Docente:** rappresenta un docente registrato nel sistema. Estende Utente e permette di gestire i vincoli orari del docente. Ogni docente può indicare al massimo tre vincoli, cioè fasce orarie in cui non è disponibile a svolgere lezione. La classe contiene inoltre un attributo che indica se il docente è anche responsabile degli orari.
- **DocenteResponsabile:** rappresenta il docente con privilegi aggiuntivi per la gestione dell'orario e delle richieste di spostamento. Il docente responsabile può visualizzare le richieste inviate dai docenti e decidere se approvarle o rifiutarle.

3.2 Gestione Didattica

- **Aula:** rappresenta un'aula disponibile per lo svolgimento delle lezioni. Ogni aula è identificata da un nome, ad esempio "Aula A1".
- **Insegnamento:** rappresenta un insegnamento attivo nell'anno accademico. Ogni insegnamento è caratterizzato da nome, numero di CFU, anno di corso e docente titolare.
- **Lezione:** rappresenta una lezione presente nell'orario settimanale. Ogni lezione è associata a un giorno, a un'ora di inizio, a un'ora di fine, a un insegnamento e a un'aula.

3.3 Gestione dei Vincoli e delle Richieste

- **VincoloDocente:** rappresenta una fascia oraria in cui un docente non è disponibile. Contiene il giorno, l'ora di inizio e l'ora di fine del vincolo. Questa classe permette di modellare le indisponibilità dei docenti.

- *RichiestaSpostamento*: rappresenta una richiesta inviata da un docente per modificare il giorno e l'orario di una lezione. Contiene la lezione da spostare, il nuovo giorno proposto, la nuova ora di inizio, la nuova ora di fine e lo stato della richiesta.

- *StatoRichiesta*: enumerazione che rappresenta i possibili stati di una richiesta di spostamento. Una richiesta nasce nello stato IN_ATTESA e può successivamente diventare APPROVATA o RIFIUTATA in base alla decisione del docente responsabile.

4. Il Package *controller*

Il package controller contiene le classi che coordinano il comportamento dell'applicazione.

- *Main*: rappresenta il punto di avvio del programma. Crea la schermata principale GUIhome, inizializza il Controller e rende visibile la finestra iniziale.

- *Controller*: rappresenta la classe centrale del flusso applicativo. Gestisce gli eventi generati dall'interfaccia grafica, effettua il login degli utenti tramite i DAO e apre la schermata corretta in base al ruolo autenticato.

Il Controller svolge il ruolo di intermediario tra la GUI e il resto del sistema. In questo modo le classi grafiche non accedono direttamente al database, ma delegano le operazioni alla logica di controllo e ai DAO. Questo migliora la separazione delle responsabilità e rende il codice più organizzato.

Le principali operazioni gestite dal Controller sono:

- selezione del tipo di utente;
- login di studente, docente e docente responsabile;
- apertura della dashboard corretta;
- visualizzazione dell'orario;
- invio di una richiesta di spostamento lezione;
- gestione delle richieste da parte del docente responsabile;
- logout e ritorno alla schermata iniziale.

5. Il Package *gui*

Il package gui contiene le classi dedicate all'interfaccia grafica, realizzata con Swing e IntelliJ IDEA GUI Designer. Le classi di questo package permettono agli utenti di interagire con il sistema tramite finestre, pulsanti e campi di input.

5.1 Schermate Principali

- *GUIhome*: rappresenta la schermata iniziale dell'applicazione. Permette all'utente di scegliere il tipo di accesso tra studente, docente e docente responsabile. Contiene inoltre i campi per inserire email, password e, nel caso del docente responsabile, il codice di accesso.
- *GUIDocente*: rappresenta l'area riservata del docente. Da questa schermata il docente può visualizzare il proprio orario, aprire il pannello per richiedere lo spostamento di una lezione ed effettuare il logout.
- *GUIstudente*: rappresenta l'area riservata dello studente. Permette allo studente di visualizzare l'orario delle lezioni e tornare alla schermata precedente.
- *GUIDocenteresponsabile*: rappresenta l'area riservata del docente responsabile. Permette di visualizzare le richieste di spostamento lezioni inviate dai docenti ed effettuare il logout.

5.2 Schermate Operative

- *GUIrichiestaspostamentolezione*: rappresenta il pannello utilizzato dal docente per inviare una richiesta di spostamento lezione. Il pannello raccoglie la data, l'ora di inizio e l'ora di fine proposte per la nuova collocazione della lezione.

I file .form associati alle classi GUI sono generati da IntelliJ IDEA GUI Designer e definiscono la disposizione grafica dei componenti Swing.

6. Il Package dao

Il package dao contiene le interfacce che definiscono le operazioni di accesso ai dati. L'utilizzo del pattern DAO consente di separare la logica applicativa dalle query SQL e dai dettagli specifici del database.

- *DocenteDAO*: definisce le operazioni di accesso ai dati relative ai docenti. Espone il metodo loginDocente, utilizzato per autenticare sia il docente normale sia il docente responsabile tramite email e password.
- *StudenteDAO*: definisce le operazioni di accesso ai dati relative agli studenti. Espone il metodo loginStudente, utilizzato per autenticare uno studente tramite email e password.

Le interfacce DAO stabiliscono quali operazioni sono disponibili, senza specificare come vengono realizzate a livello SQL. Questo permette di mantenere il codice più flessibile e più facile da modificare.

7. Il Package *implementazioneDao*

Il package *implementazioneDao* contiene le implementazioni concrete delle interfacce definite nel package *dao*. Le classi presenti in questo package comunicano direttamente con il database PostgreSQL tramite JDBC.

- *DocentePostgresDAO*: implementa l'interfaccia *DocenteDAO*. Esegue una query sulle tabelle *utente* e *docente* per verificare le credenziali del docente. Se il login va a buon fine, costruisce e restituisce un oggetto *Docente* contenente nome, cognome, email, password e informazione sul ruolo di responsabile.

- *StudentePostgresDAO*: implementa l'interfaccia *StudenteDAO*. Esegue una query sulle tabelle *utente* e *studente* per verificare le credenziali dello studente. Se il login è corretto, restituisce un oggetto *Studente* contenente nome, cognome, email, password, matricola e anno di corso.

L'uso di *PreparedStatement* consente di eseguire query parametrizzate, separando i valori inseriti dall'utente dalla struttura della query SQL.

8. Il Package *database_connection*

Il package *database_connection* contiene la classe responsabile della connessione al database PostgreSQL.

- *ConnessioneDatabase*: gestisce la connessione JDBC al database. La classe utilizza il pattern Singleton, mantenendo una sola istanza condivisa della connessione. I DAO utilizzano questa connessione per eseguire le query necessarie al login degli utenti.

La separazione della connessione in un package dedicato permette di centralizzare le informazioni necessarie per collegarsi al database, come URL, utente e password, rendendo il progetto più ordinato.

9. Funzionalità Principali

Le funzionalità principali implementate nel sistema sono:

- Login *studente*: lo studente inserisce email e password nella schermata iniziale. Il Controller richiama *StudenteDAO* e, se le credenziali sono corrette, apre la schermata *GUIstudente*.

- Login *docente*: il docente inserisce email e password. Il Controller richiama *DocenteDAO* e, in caso di autenticazione corretta, apre la schermata *GUIDocente*.

- Login *docente responsabile*: il docente responsabile inserisce email, password e codice responsabile. Se i dati sono corretti, il sistema apre la schermata GUI *docenteresponsabile*.
- Visualizzazione orario: studenti e docenti possono visualizzare l'orario delle lezioni tramite la propria area riservata.
- Richiesta di spostamento lezione: il docente può aprire un form dedicato e indicare il nuovo giorno, la nuova ora di inizio e la nuova ora di fine desiderate per una lezione.
- Gestione richieste: il docente responsabile può visualizzare le richieste di spostamento inviate dai docenti e decidere se approvarle o rifiutarle.

10. Documentazione Interna del Codice

Il progetto è stato documentato internamente tramite commenti Javadoc. La documentazione è stata inserita nelle classi pubbliche, nei costruttori e nei metodi principali.

La Javadoc descrive il ruolo delle classi, il comportamento dei metodi, i parametri ricevuti e i valori restituiti. Questo rende il codice più leggibile e facilita la comprensione del progetto da parte di altri sviluppatori e durante la valutazione.

Sono stati documentati in particolare:

- le classi del package `model`;
- le schermate principali del package `gui`;
- il Controller e la classe `Main`;
- le interfacce DAO;
- le implementazioni PostgreSQL dei DAO;
- la classe di connessione al database.

11. Qualità del Codice

Il codice è stato organizzato in package separati per distinguere chiaramente le responsabilità delle diverse parti del sistema. Questa scelta riduce l'accoppiamento tra interfaccia grafica, logica applicativa e accesso ai dati.

L'utilizzo del pattern DAO permette di isolare le operazioni verso il database dalla logica del Controller. Le query SQL sono contenute nelle classi del package `implementazione dao`, mentre il resto dell'applicazione lavora con oggetti del `model`.

Inoltre, l'utilizzo di PreparedStatement nelle classi DAO rende più ordinata e sicura la gestione delle query SQL. La presenza della Javadoc contribuisce alla qualità complessiva del codice, migliorandone leggibilità e manutenibilità.

12. Diagrammi di Sequenza

Nel progetto sono stati realizzati diagrammi di sequenza per rappresentare alcune funzionalità fondamentali del sistema.

Un primo diagramma può rappresentare il login di uno studente o di un docente: l'utente inserisce le credenziali nella GUIhome, il Controller riceve l'evento, richiama il DAO corrispondente, il DAO interroga il database e restituisce l'oggetto utente se le credenziali sono corrette. Infine, il Controller apre la schermata corretta.

Un secondo diagramma può rappresentare la richiesta di spostamento lezione: il docente accede alla propria dashboard, apre il pannello di richiesta, inserisce giorno e orario proposti, il Controller crea una RichiestaSpostamento e la memorizza nello stato IN_ATTESA. Successivamente, il docente responsabile può visualizzare la richiesta e approvarla o rifiutarla.

Questi diagrammi mostrano l'interazione tra classi appartenenti a package diversi, in particolare gui, controller, model, dao e implementatedao.

13. Conclusioni

Il sistema realizzato consente la gestione semplificata dell'orario delle lezioni di un corso di laurea. L'applicazione distingue tre tipologie di utenti: studente, docente e docente responsabile, offrendo funzionalità diverse in base al ruolo autenticato.

La struttura a package permette di separare chiaramente le responsabilità del progetto: il model rappresenta il dominio, la GUI gestisce l'interazione con l'utente, il controller coordina il flusso applicativo, i DAO definiscono l'accesso ai dati e le implementazioni PostgreSQL gestiscono la comunicazione con il database.

Questa organizzazione rende il progetto più leggibile, manutenibile ed estendibile. La presenza della documentazione Javadoc e l'attenzione alla qualità del codice contribuiscono a rendere il sistema più chiaro e coerente con gli obiettivi della consegna.